Trier une table

sorted() et .sort()

Pour trier en ordre croissant de façon simple et facile, il suffit d'appeler la fonction `sorted()`. Elle renvoie un nouveau tableau trié :

```
>>> sorted([5, 2, 3, 1, 4])
[1, 2, 3, 4, 5]
```

Ici on a trié un tableau de nombres entiers. On peut faire la même chose avec un tableau de nombres decimaux (`float`) ou de chaines de caractères. ⚠️ Les chaines de caractères sont triées par ordre lexicographique[1].

```
>>> sorted(['pomme', 'banane', 'orange', 'fraise'])
['banane', 'fraise', 'orange', 'pomme']
```

On peut aussi trier un p-uplet ou les clés d'un dictionnaire, la fonction renvoie toujours un tableau :

```
>>> sorted((5,2,3,1,4))
[1, 2, 3, 4, 5]
>>> sorted({'un':1, 'deux':2, 'trois':3 })
['deux', 'trois', 'un']
```

On peut aussi utiliser la méthode `.sort()` qui elle modifie le tableau (et renvoie `None` pour éviter les confusions).

```
>>> a = [5, 2, 3, 1, 4]
>>> a.sort()
>>> a
[1, 2, 3, 4, 5]
```

Une autre différence est que la méthode `.sort()` est seulement utilisées pour les tableaux. Au contraire, la fonction `sorted()` accepte n'importe quel itérable, par exemple les clé d'un dictionnaire.

```
>>> sorted({1: 'D', 2: 'B', 3: 'B', 4: 'E', 5: 'A'})
[1, 2, 3, 4, 5]
>>> {1: 'D', 2: 'B', 3: 'B', 4: 'E', 5: 'A'}.sort()
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
AttributeError: 'dict' object has no attribute 'sort'
```

On peut trier sur des types construits, dans ce cas le tri est fait par ordre des éléments. Par exemple, on peut trier une liste de p-uplets contenant les pays.

```
>>> pays = [('France', 'Paris', 68), ('Allemagne', 'Berlin', 82), ('Italie', 'Rome', 60)]
>>> sorted(pays)
[('Allemagne', 'Berlin', 82), ('France', 'Paris', 68), ('Italie', 'Rome', 60)]
```

Dans ce cas, le tri est fait par `pays[0]` (type str) donc par ordre alphabétique : `'Allemagne' < 'France'< 'Italie'`.

Par contre, on ne peut pas trier un tableau de dictionnaires :

```
>>> pays = [{'Capitale': 'Paris', 'Pays': 'France', 'Population (ml)': 68},
    {'Capitale': 'Berlin', 'Pays': 'Allemagne', 'Population (ml)': 82},
    {'Capitale': 'Rome', 'Pays': 'Italie', 'Population (ml)': 60}]
>>> sorted(pays)
Traceback (most recent call last):
  File "<interactive input>", line 1, in <module>
TypeError: '<' not supported between instances of 'dict' and 'dict'
```

Paramètre key

`.sort()` et `sorted()` acceptent un paramètre nommé `key` permettant de spécifier une fonction à appeler sur chaque élément du tableau afin d'effectuer des comparaisons. Par exemple, on peut modifier l'ordre de tri d'un tableau de nombres au format `str` :

```
>>> sorted(['5', '3', '1', '11', '21'])
['1', '11', '21', '3', '5']
```

en précisant que les données doivent être converties en entier par la fonction `int()` avant d'être triées :

```
>>> sorted(['5', '3', '1', '11', '21'], key=int)
['1', '3', '5', '11', '21']
```

De la même façon, le paramètre `key` permet de trier une table en précisant les colonnes selon lesquelles on veut trier. Par exemple, si on veut trier le tableau de p-uplets des pays selon leur population :

```
>>> pays = [('France', 'Paris', 68), ('Allemagne', 'Berlin', 82), ('Italie', 'Rome', 60)]
```

On peut utiliser une fonction `popul` qui renvoie la population :

```
def popul(x):
    return x[2]
```

et qui sert de clé de `sorted()` :

```
>>> sorted(pays, key=popul)
[('Italie', 'Rome', 60), ('France', 'Paris', 68), ('Allemagne', 'Berlin', 82)]
```

Ou bien l'écrire directement dans une fonction lambda :

```
>>> sorted(pays, key=lambda x:x[2])
[('Italie', 'Rome', 60), ('France', 'Paris', 68), ('Allemagne', 'Berlin', 82)]
```

Pour trier selon plusieurs colonnes, il suffit que la fonction de tri renvoie un p-uplet, par exemple les pays par population et par capital on fera :

```
>>> pays = [('France', 'Paris', 68), ('Allemagne', 'Berlin', 82), ('Italie', 'Rome', 60),
('Royaume-Uni', 'Londres', 68),]
>>> sorted(pays, key=lambda x:(x[2], x[1]))
[('Italie', 'Rome', 60),
```

```
 ('Royaume-Uni', 'Londres', 68),
 ('France', 'Paris', 68),
 ('Allemagne', 'Berlin', 82)
 ```

A noter que dans ce cas la fonction `lambda x:x[2]` renvoie un itérable (tableau des populations) qui est trié par la fonction `sorted()`. De la même façon, on peut ainsi trier un tableau de dictionnaires en ordre croissant de population (alors qu'on ne pouvait pas le faire sans le paramètre `key`) :

```` py
>>> pays = [{'Capitale': 'Paris', 'Pays': 'France', 'Population (ml)': 68}, \
            {'Capitale': 'Berlin', 'Pays': 'Allemagne', 'Population (ml)': 82}, \
            {'Capitale': 'Rome', 'Pays': 'Italie', 'Population (ml)': 60}]

>>> sorted(pays, key= lambda x:x['Population (ml)'])
[{'Capitale': 'Rome', 'Pays': 'Italie', 'Population (ml)': 60},
 {'Capitale': 'Paris', 'Pays': 'France', 'Population (ml)': 68},
 {'Capitale': 'Berlin', 'Pays': 'Allemagne', 'Population (ml)': 82}]
```

## Paramètre reverse

`.sort()` et `sorted()` acceptent aussi un paramètre nommé `reverse` avec une valeur booléenne. Par défaut, `reverse` est `False`, c'est-à-dire qu'on tri en ordre croissant, mais on peut le changer pour indiquer un ordre décroissant des tris. Par exemple, pour avoir les pays dans par population décroissante :

```
>>> sorted(pays, key = lambda x:x[2], reverse = True)
[('Allemagne', 'Berlin', 82), ('France', 'Paris', 68), ('Italie', 'Rome', 60)]
```

**Exercice corrigé**

En utilisant le tableau de dictionnaires des codes postaux, écrire une fonction qui renvoie la commune qui a le plus petit code postal, le plus grand code postal, la plus grande longitude, la plus éloignée de Manosque. Note : La distance en mètres entre les points de coordonnées ( ; ) et ( ; ) est donnée par la formule de Pythagore :

- $\sqrt{\phantom{xxxxxxxxxxxx}}$
- $\sqrt{\phantom{xxxxxx}}$
- 
- 

Source : http://villemin.gerard.free.fr/aGeograp/Distance.htm

**Réponse**

1. On commence par comparer les codes Unicode du premier caractère de chaque chaîne, puis en cas d'égalité le second caractère, et ainsi de suite comme dans un dictionnaire. Attention aux majuscules et aux nombres, '11' est plus petit que '2' ! ↩